

Warm up

What will the following snippet of code print?

```
#include <stdio.h>

int main(void) {
    unsigned int i = 5;
    int j = -1;
    if (i > j) {
        printf("i > j\n");
    } else {
        printf("i <= j\n");
    }
    return 0;
}
```

- Answer: $i \leq j$
 - When comparing i and j , the compiler will treat both values as unsigned integers, since i is unsigned (by C standard)
- Unsigned comparison in assembly:

```
cmp     w1, w0    // i is in w1, j is in w0
bls     .L2       // bls performs unsigned comparison
```

Precept 19: ARMv8 function calls

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

April 9, 2026

Overview

- 1 ARMv8 function call conventions
- 2 Your turn: writing assembly for functions
- 3 Summary

Function call conventions

- A calling convention is a set of rules that govern how functions receive parameters and return values
- It answers questions like:
 - How are arguments passed to functions (e.g., via registers or stack)?
 - Where should the function return to after execution?
 - How are return values passed back to the caller?
 - Which registers are preserved across function calls?
- These are effectively an agreement between the caller and callee on how to communicate during a function call
- Note: different architectures and operating systems may have different calling conventions, since they may have different registers, architectural features, etc.

Recall: general-purpose registers

- ARMv8 has 31 general-purpose registers (r0-r30)
- In code, registers are referred to as xN or wN (e.g., x0, w0 for r0)
 - x0–x30 refers to the full 64-bit registers
 - w0–w30 refers to the lower 32 bits of the registers
- There are conventions for how registers are used in procedure calls:
 - r0: used to return a value from a function
 - r0–r7: used for passing arguments to functions, and for temporary values within functions
 - r8: indirect result location parameter (we will not use)
 - r9–r15: used for temporary values within functions
 - r16–r17: intra-call scratch registers (we will not use)
 - r18: platform register (we will not use)
 - r19–r28: used for optimised program design
 - r29: frame pointer (we will not use)
 - r30: link register, used to store return address

Rule 1: return address in x30

- The return address is stored in the link register x30
- The `bl` instruction stores the address of the instruction immediately following `bl` in x30
- To return from a function, we can use the instruction `ret`, which jumps to the address stored in x30

```
bl foo    // reg[x30] <- PC + 4; PC <- foo
ret       // PC <- reg[x30]
```

- See *Trace of an ARMv8 Assembly Language Function Call*
- In the function prologue, callee can save the return address (e.g., by pushing x30 onto the stack)
 - Why would the callee want to save the return address? If the callee needs to call another function, it will overwrite x30 with the new return address, so it must save the original return address
 - If function is a *leaf* function (i.e., it does not call any other functions), then this does not need to be done
- Then, in the function epilogue, the callee can restore the return address (e.g., by popping it from the stack back into x30) before `ret`

Rules 2-3: function arguments in x0-x7, return value in x0

- The first 8 arguments to a function are passed, in order, in registers x0-x7
 - If a function has more than 8 arguments, additional arguments are passed on the stack (we will not cover this case)
- The return value of a function is stored in register x0
- Example: `int result = foo(a, b)`
 - The value of a is stored in x0 before the call
 - The value of b is stored in x1 before the call
 - After the call, the return value of foo is stored in x0
- See `euclid.c` and `euclid.s`
- Make sure `sp` always contains an address that is evenly divisible by 16
 - Required for stack alignment
- Use immediate offset memory addressing to load / store local variables on the stack
 - Offsets are set relative to stack pointer at the beginning of the function

Can we do better?

- What are the pros and cons of the function structure in `euclid.s`?
 - Pros: conceptually simple, generalisable to any function
 - Cons: inefficient, since multiple loads and stores to the stack are required to save and restore registers
- Can we do better?
 - Recall: registers are much faster to access than memory (load / store)
 - If we can avoid saving and restoring registers to the stack, we can improve performance
- Compiler will produce something similar to `euclid.s` without optimisations

Rule 4: caller- and callee-saved registers

- Registers x0–x7, x9–x15 are *caller-saved* registers
 - Caller is responsible for saving and restoring their values if it needs to use them after the function call
 - Therefore, callee is free to use these registers without worrying about preserving their values
- Registers x19–x28 are *callee-saved* registers
 - Callee is responsible for saving and restoring their values if it uses them
 - Therefore, caller can assume that their values will be preserved across the function call
- Insight: use callee-saved registers to avoid unnecessary loads and stores to the stack, which can improve performance
- See `euclidopt.s`
- `LABSSECOND .req x23` defines `LABSSECOND` as an alias for `x23`
- In function prologue, callee can save x30 and callee-saved registers it plans to use onto the stack
- In function epilogue, callee can restore x30 and callee-saved registers from the stack before `ret`

Did we do better?

- What are the pros and cons of the function structure in `euclidopt.s`?
 - Pros: fewer loads and stores → faster
 - Cons:
 - ◊ What if we had more than 10 parameters and locals?
 - ◊ What if our local variables are too large to fit in registers (e.g., arrays or structs)?
 - ◊ What if we wanted to get the address of a local variables (e.g., `&x`)?
 - ◊ Can we perform the same optimisations for `main` in `euclidopt.s`?
No, since `scanf` needs `&l1` and `&l2` as arguments, which requires storing `l1` and `l2` on the stack
- Compiler will produce something similar to `euclidopt.s` with optimisations enabled

Overview

- 1 ARMv8 function call conventions
- 2 Your turn: writing assembly for functions
- 3 Summary

rectGlobal.s

```
.section .bss
iLength: .skip 4
iWidth: .skip 4
iPerim: .skip 4

.section .text
// Must be a multiple of 16
.equ MAIN_STACK_BYTECOUNT, 16
.global main

main:
// Prolog
sub sp, sp, MAIN_STACK_BYTECOUNT
str x30, [sp]

// ... body of main ...

// Epilog
mov w0, 0
ldr x30, [sp]
add sp, sp, MAIN_STACK_BYTECOUNT
ret
```

What changes do we need to make to the above assembly code to implement a version of the code that does not use global variables?

Possible solution: rect.s

```
.section .bss
// no globals

.section .text
// Must be a multiple of 16
.equ    MAIN_STACK_BYTECOUNT, 32

// Offsets for local variables
.equ    iLength, 8
.equ    iWidth, 12
.equ    iPerim, 16

.global main

main:
    // Prolog
    sub    sp, sp, MAIN_STACK_BYTECOUNT
    str    x30, [sp]

    // ... body of main ...

    // Epilog
    mov    w0, 0
    ldr    x30, [sp]
    add    sp, sp, MAIN_STACK_BYTECOUNT
    ret
```

- Previously, iLength was the address of a global variable, so we had to use ldr and str to access it
- Now, iLength is an offset from the stack pointer, so we can use register immediate addressing to access it (e.g., ldr w1, [sp, iLength])
- To get the address of iLength, we can use add x1, sp, iLength

Overview

- 1 ARMv8 function call conventions
- 2 Your turn: writing assembly for functions
- 3 Summary

Summary

- ARMv8 function call conventions specify how functions receive parameters and return values and how the caller and callee should manage registers during function calls
- Dates to remember:
 - Assignment 4 Quiz: Mon, 4/13 in lecture
 - Assignment 5: due Tue, 4/21 at 9pm
 - Assignment 5 Quiz: Wed, 4/22 in lecture
- Acknowledgements
 - Most of the code in these slides is adapted from the precept materials, written by Prof. Bob Dondero and William Ughetta
 - Can be accessed at <https://www.cs.princeton.edu/courses/archive/spring26/cos217/precepts/w11p2/>